Lecture 2 - SQL Fundamentals

DDL, DML, DQL — Table Creation, Constraints & Data Types

Fettah KIRAN
Department of AI and Data Engineering
Yıldız Technical University
fkiran@yildiz.edu.tr

📋 Today's Agenda (1:00 PM – 3:45 PM)

Time	Topic	Type
1:00 – 1:10	Week 1 recap quiz	Review
1:10 – 1:40	SQLite Data Types & Type Affinity	**Theory** 📖
1:40 – 2:10	Constraints Deep Dive	**Theory** 📖
2:10 – 2:25	☕ Break (15 min)	
2:25 – 2:45	CREATE TABLE Patterns	**Theory** 📖
2:45 – 3:05	DML in Depth: INSERT, UPDATE, DELETE	**Theory** 📖
3:05 – 3:20	Built-in Functions (String, Math, Date)	**Theory** 📖
3:20 – 3:40	Hands-on Coding Practice	**Practice** 💻
3:40 – 3:45	Recap & next week preview	Wrap-up

Week 1 Quick Recap

Concept	Key Point
Database vs Spreadsheet	Scalable, multi-user, strict types
SQLite + VS Code	Serverless, single-file, `sqlite3 longlist.db`
SELECT / LIMIT	Retrieve columns; limit output rows
WHERE	Filter rows with conditions (`=` , `!=` , `<` , `>`)
LIKE	Pattern matching (`%` any chars, `_` one char)
NULL	Unknown/missing; use `IS NULL` / `IS NOT NULL`
BETWEEN / IN	Range and set filtering
ORDER BY	Sort results (ASC default, DESC)
Aggregates	`COUNT` , `SUM` , `AVG` , `MIN` , `MAX` , `ROUND`
DISTINCT / AS	Remove duplicates; alias columns

🔗 From Querying to Building

Last week we **queried** an existing database (`longlist.db`):

```
SELECT title, author FROM longlist WHERE year = 2023 ORDER BY rating DESC LIMIT 5;
```

> **This week's big question:** How was that `longlist` table created in the first place?

Today we go **behind the scenes**:

- **Data Types** — What kinds of values can columns hold?
- **Constraints** — How do we enforce data integrity?
- **DDL** — `CREATE TABLE`, `ALTER TABLE`, `DROP TABLE`
- **DML** — `INSERT`, `UPDATE`, `DELETE` (writing data, not just reading)
- **Built-in Functions** — String, numeric, and date operations

SQLite Data Types & Type Affinity

SQLite Type System: The Basics

SQLite uses **5 storage classes** (not data types!):

Storage Class	Description	Size
`NULL`	Missing or unknown value	0 bytes
`INTEGER`	Signed integer	1, 2, 3, 4, 6, or 8 bytes
`REAL`	8-byte IEEE floating point	8 bytes
`TEXT`	UTF-8, UTF-16BE or UTF-16LE string	Variable
`BLOB`	Binary data, stored exactly as input	Variable

> 💡 SQLite is **dynamically typed** — the type belongs to the **value**, not the column!

Type Affinity in SQLite

When you declare a column type, SQLite maps it to an **affinity**:

Declared Type Contains	Affinity	Examples
`INT`	INTEGER	`INT`, `INTEGER`, `BIGINT`, `SMALLINT`
`CHAR`, `TEXT`, `CLOB`	TEXT	`VARCHAR(50)`, `TEXT`, `CHAR(10)`
`BLOB` or none	BLOB	`BLOB`, no type specified
`REAL`, `FLOAT`, `DOUBLE`	REAL	`REAL`, `FLOAT`, `DOUBLE`
Everything else	NUMERIC	`DECIMAL(10,2)`, `BOOLEAN`, `DATE`

```sql
-- All of these are valid in SQLite!
CREATE TABLE demo (
    a INTEGER,          -- INTEGER affinity
    b VARCHAR(100),     -- TEXT affinity (ignores the 100)
    c DECIMAL(10,2),    -- NUMERIC affinity
    d BOOLEAN,          -- NUMERIC affinity (0/1)
    e DATE              -- NUMERIC affinity
);
```

Type Affinity: How It Works

```sql
-- SQLite is flexible about types
CREATE TABLE flex_demo (val NUMERIC);

INSERT INTO flex_demo VALUES (42);          -- stored as INTEGER
INSERT INTO flex_demo VALUES (3.14);        -- stored as REAL
INSERT INTO flex_demo VALUES ('hello');      -- stored as TEXT
INSERT INTO flex_demo VALUES (NULL);        -- stored as NULL
INSERT INTO flex_demo VALUES (X'CAFE');     -- stored as BLOB

-- Check actual storage type with typeof()
SELECT val, typeof(val) FROM flex_demo;
```

val	typeof(val)
42	integer
3.14	real
hello	text
NULL	null
(binary)	blob

Comparison: MySQL vs SQLite Types

MySQL	SQLite Equivalent	Notes
`TINYINT`, `SMALLINT`, `INT`, `BIGINT`	`INTEGER`	Auto-sizes (1–8 bytes)
`CHAR(n)`, `VARCHAR(n)`, `TEXT`	`TEXT`	No length enforcement
`FLOAT`, `DOUBLE`, `DECIMAL(p,s)`	`REAL`	No exact decimal type
`DATE`, `DATETIME`, `TIMESTAMP`	`TEXT` or `INTEGER`	ISO-8601 string or Unix epoch
`BOOLEAN`	`INTEGER`	0 = false, 1 = true
`ENUM('a','b','c')`	`TEXT + CHECK`	Use CHECK constraint
`AUTO_INCREMENT`	`INTEGER PRIMARY KEY`	Automatic in SQLite
`BLOB`	`BLOB`	Same

> 📝 **Exam tip:** Know the MySQL data types and their purposes, plus how SQLite differs.

🧠 Exam Focus: Data Types

> **Q:** Which data type should be used to store a monetary value like 12345.67 with exact precision?
>
> A) FLOAT
> B) DOUBLE
> C) **DECIMAL(7,2)** ✅
> D) INT

💡 In MySQL: `DECIMAL(p,s)` provides **exact** precision (p = total digits, s = decimals).
💡 In SQLite: Use `REAL` (approximate) or store as `INTEGER` in cents (12345.67 → 1234567).

Best Practices for SQLite Data Types

Dates

```sql
-- Store as TEXT in ISO-8601 format
CREATE TABLE events (
    event_date TEXT DEFAULT (date('now'))  -- '2025-03-01'
);

-- Or as INTEGER (Unix timestamp)
CREATE TABLE logs (
    created_at INTEGER DEFAULT (strftime('%s', 'now'))
);
```

Booleans

```sql
-- Use INTEGER with CHECK
CREATE TABLE tasks (
    is_completed INTEGER DEFAULT 0 CHECK (is_completed IN (0, 1))
);
```

Money

```sql
-- Store in cents as INTEGER to avoid floating-point errors
CREATE TABLE products (
    price_cents INTEGER NOT NULL CHECK (price_cents >= 0)
);
-- $12.99 → 1299
```

Constraints Deep Dive

Constraints Overview

Constraints enforce **data integrity** — they prevent bad data from entering the database.

Constraint	Purpose	Level
`PRIMARY KEY`	Uniquely identifies each row	Column / Table
`NOT NULL`	Prevents NULL values	Column
`UNIQUE`	No duplicate values allowed	Column / Table
`CHECK`	Validates against a condition	Column / Table
`DEFAULT`	Fallback value when none given	Column
`FOREIGN KEY`	Links to another table's PK	Table

> 💡 Constraints can be defined **inline** (column-level) or as **table-level** constraints.

PRIMARY KEY

```sql
-- Method 1: Column-level (most common in SQLite)
CREATE TABLE student (
    student_id INTEGER PRIMARY KEY,  -- auto-increments!
    name       TEXT NOT NULL
);

-- Method 2: Table-level (required for composite PKs)
CREATE TABLE enrollment (
    student_id INTEGER,
    course_id  INTEGER,
    semester   TEXT,
    PRIMARY KEY (student_id, course_id, semester)  -- Composite PK
);
```

Rules:

- **One** primary key per table (can be composite)
- Implicitly `NOT NULL` and `UNIQUE`
- In SQLite: `INTEGER PRIMARY KEY` = auto-incrementing rowid
- `AUTOINCREMENT` keyword prevents reuse of deleted IDs

INTEGER PRIMARY KEY vs AUTOINCREMENT

```sql
-- Without AUTOINCREMENT: IDs may be reused
CREATE TABLE t1 (id INTEGER PRIMARY KEY, name TEXT);
INSERT INTO t1 (name) VALUES ('Alice');  -- id = 1
INSERT INTO t1 (name) VALUES ('Bob');    -- id = 2
DELETE FROM t1 WHERE id = 2;
INSERT INTO t1 (name) VALUES ('Carol');  -- id = 2 (reused!)

-- With AUTOINCREMENT: IDs never reused
CREATE TABLE t2 (id INTEGER PRIMARY KEY AUTOINCREMENT, name TEXT);
INSERT INTO t2 (name) VALUES ('Alice');  -- id = 1
INSERT INTO t2 (name) VALUES ('Bob');    -- id = 2
DELETE FROM t2 WHERE id = 2;
INSERT INTO t2 (name) VALUES ('Carol');  -- id = 3 (never reuses 2)
```

> 💡 `AUTOINCREMENT` is slightly slower. Use it only when ID reuse is unacceptable.

🧠 Exam Focus: AUTO_INCREMENT

> **Q:** What is the effect of the AUTO_INCREMENT attribute on a column?
>
> A) It fills the column with zeros automatically
>
> B) **It generates a unique integer value for each new row** ✅
>
> C) It creates an index on the column
>
> D) It sets the column default to 1

💡 In MySQL: `AUTO_INCREMENT` · In SQLite: `INTEGER PRIMARY KEY` (automatic) or `AUTOINCREMENT`

NOT NULL Constraint

```sql
CREATE TABLE student (
    student_id INTEGER PRIMARY KEY,
    first_name TEXT NOT NULL,       -- Cannot be NULL
    last_name  TEXT NOT NULL,       -- Cannot be NULL
    nickname   TEXT                 -- Can be NULL (optional)
);

-- This works:
INSERT INTO student (first_name, last_name) VALUES ('Alice', 'Yılmaz');

-- This FAILS:
INSERT INTO student (first_name) VALUES ('Bob');
-- ERROR: NOT NULL constraint failed: student.last_name
```

When to use NOT NULL:

- Required fields (name, email, etc.)
- Always when a value **must** exist
- Primary keys are implicitly NOT NULL

UNIQUE Constraint

```sql
CREATE TABLE student (
    student_id INTEGER PRIMARY KEY,
    email      TEXT UNIQUE NOT NULL,   -- No two students share an email
    phone      TEXT UNIQUE             -- Unique but allows ONE NULL
);

-- Table-level UNIQUE (composite uniqueness)
CREATE TABLE enrollment (
    student_id INTEGER,
    course_id  INTEGER,
    semester   TEXT,
    UNIQUE(student_id, course_id, semester)  -- Same student can't enroll twice
);
```

Key Points:

- UNIQUE allows **one NULL** (NULL ≠ NULL in SQL)
- PRIMARY KEY = UNIQUE + NOT NULL
- Multiple UNIQUE constraints per table are allowed

🧠 Exam Focus: UNIQUE Constraint

> **Q:** Which constraint ensures that no two rows can have the same value in a specified column?
>
> A) NOT NULL
> B) PRIMARY KEY
> C) **UNIQUE** ✅
> D) DEFAULT

💡 Both PRIMARY KEY and UNIQUE prevent duplicates, but:

- **PRIMARY KEY** also implies NOT NULL, and there's only one per table
- **UNIQUE** allows NULL and you can have multiple UNIQUE constraints

CHECK Constraint

```sql
CREATE TABLE student (
    student_id INTEGER PRIMARY KEY,
    first_name TEXT NOT NULL,
    age        INTEGER CHECK (age >= 16 AND age <= 100),
    gpa        REAL CHECK (gpa >= 0.0 AND gpa <= 4.0),
    status     TEXT CHECK (status IN ('active', 'inactive', 'graduated'))
);

-- This works:
INSERT INTO student (first_name, age, gpa, status)
VALUES ('Alice', 20, 3.75, 'active');

-- This FAILS:
INSERT INTO student (first_name, age, gpa, status)
VALUES ('Bob', 20, -1.0, 'active');
-- ERROR: CHECK constraint failed
```

🧠 Exam Focus: CHECK Constraint

> **Q:** Consider: `CREATE TABLE t (id INT, val INT CHECK (val > 0))`. Which INSERT will fail?
>
> A) `INSERT INTO t VALUES (1, 5)`
> B) `INSERT INTO t VALUES (2, 100)`
> C) **`INSERT INTO t VALUES (3, -1)`** ✅
> D) `INSERT INTO t VALUES (4, 1)`

💡 The CHECK constraint `val > 0` rejects any value ≤ 0.

DEFAULT Constraint

```sql
CREATE TABLE student (
    student_id   INTEGER PRIMARY KEY,
    first_name   TEXT NOT NULL,
    gpa          REAL DEFAULT 0.0,
    status       TEXT DEFAULT 'active',
    enrolled_at  TEXT DEFAULT (datetime('now')),  -- Dynamic default!
    is_active    INTEGER DEFAULT 1
);

-- Omitting columns with defaults
INSERT INTO student (first_name) VALUES ('Alice');
-- gpa = 0.0, status = 'active', enrolled_at = current datetime, is_active = 1
```

Dynamic Defaults in SQLite:

```sql
-- Use parentheses for expressions
created_at TEXT DEFAULT (date('now'))
updated_at TEXT DEFAULT (datetime('now'))
random_id  TEXT DEFAULT (hex(randomblob(4)))
```

FOREIGN KEY Constraint

```sql
CREATE TABLE department (
    dept_id   INTEGER PRIMARY KEY,
    dept_name TEXT NOT NULL
);

CREATE TABLE student (
    student_id INTEGER PRIMARY KEY,
    name       TEXT NOT NULL,
    dept_id    INTEGER,
    FOREIGN KEY (dept_id) REFERENCES department(dept_id)
        ON DELETE SET NULL     -- If dept deleted, set student's dept to NULL
        ON UPDATE CASCADE      -- If dept_id changes, update in student too
);
```

ON DELETE / ON UPDATE Actions:

Action	Behavior
`CASCADE`	Delete/update child rows automatically
`SET NULL`	Set FK to NULL when parent deleted
`SET DEFAULT`	Set FK to default value
`RESTRICT`	Prevent deletion if children exist
`NO ACTION`	Same as RESTRICT (default)

Foreign Keys: Complete Example

```sql
PRAGMA foreign_keys = ON;  -- Must enable!
CREATE TABLE department (
    dept_id INTEGER PRIMARY KEY,
    name    TEXT NOT NULL
);

CREATE TABLE student (
    student_id INTEGER PRIMARY KEY,
    name       TEXT NOT NULL,
    dept_id    INTEGER,
    FOREIGN KEY (dept_id) REFERENCES department(dept_id) ON DELETE CASCADE
);

INSERT INTO department VALUES (1, 'Math'), (2, 'CS');
INSERT INTO student VALUES (1, 'Alice', 1), (2, 'Bob', 1), (3, 'Carol', 2);

-- Delete department 1 → cascades to students
DELETE FROM department WHERE dept_id = 1;
SELECT * FROM student;  -- Alice and Bob are gone!
```

Constraints Summary Table

Constraint	What It Does	Example
`PRIMARY KEY`	Unique row identifier	`student_id INTEGER PRIMARY KEY`
`NOT NULL`	No empty values	`name TEXT NOT NULL`
`UNIQUE`	No duplicates	`email TEXT UNIQUE`
`CHECK`	Validate condition	`CHECK (age >= 18)`
`DEFAULT`	Fallback value	`DEFAULT 0.0`
`FOREIGN KEY`	Link to another table	`REFERENCES dept(id)`

> 📝 **Exam tip:** Know the purpose of each constraint and what SQL errors they prevent.

DML in Depth

INSERT Patterns

```sql
-- Pattern 1: Specify columns (recommended)
INSERT INTO student (first_name, last_name, email)
VALUES ('Alice', 'Yılmaz', 'alice@ytu.edu.tr');
-- Pattern 2: All columns (fragile — breaks if schema changes)
INSERT INTO student VALUES (1, 'Alice', 'Yılmaz', 'alice@ytu.edu.tr', NULL, 0.0, NULL);

-- Pattern 3: Multiple rows
INSERT INTO student (first_name, last_name, email) VALUES
    ('Bob',   'Kaya',  'bob@ytu.edu.tr'),
    ('Carol', 'Demir', 'carol@ytu.edu.tr');

-- Pattern 4: INSERT with SELECT (copy data)
INSERT INTO archive_students (first_name, last_name, gpa)
SELECT first_name, last_name, gpa
FROM student
WHERE gpa < 2.0;

-- Pattern 5: INSERT OR REPLACE (upsert in SQLite)
INSERT OR REPLACE INTO student (student_id, first_name, last_name, email)
VALUES (1, 'Alice', 'Yılmaz-Updated', 'alice.new@ytu.edu.tr');
```

INSERT OR ... (SQLite-specific Conflict Resolution)

```sql
-- INSERT OR IGNORE: Skip if constraint violated
INSERT OR IGNORE INTO student (first_name, last_name, email)
VALUES ('Alice', 'Yılmaz', 'alice@ytu.edu.tr');
-- If email exists, silently skips (no error)

-- INSERT OR REPLACE: Replace the existing row
INSERT OR REPLACE INTO student (student_id, first_name, last_name, email)
VALUES (1, 'Alice', 'Yılmaz', 'alice@ytu.edu.tr');
-- If PK 1 exists, deletes old row and inserts new one

-- INSERT OR ABORT: Abort the statement (default)
INSERT OR ABORT INTO student (first_name, last_name, email)
VALUES ('Alice', 'Yılmaz', 'alice@ytu.edu.tr');
-- Raises error if email exists
```

> 💡 MySQL uses `INSERT IGNORE` and `ON DUPLICATE KEY UPDATE` for similar behavior.

UPDATE Patterns

```sql
-- Basic UPDATE
UPDATE student
SET gpa = 3.80
WHERE student_id = 1;

-- Update multiple columns
UPDATE student
SET gpa = 3.85,
    email = 'alice.updated@ytu.edu.tr'
WHERE student_id = 1;

-- Conditional UPDATE with CASE
UPDATE student
SET status = CASE
    WHEN gpa >= 3.5 THEN 'honors'
    WHEN gpa >= 2.0 THEN 'good_standing'
    ELSE 'probation'
END;

-- Update with calculation
UPDATE student
SET gpa = MIN(gpa + 0.1, 4.0)  -- Boost GPA by 0.1, cap at 4.0
WHERE dept_id = 1;
```

DELETE Patterns

```sql
-- Delete specific row
DELETE FROM student WHERE student_id = 5;

-- Delete with condition
DELETE FROM student WHERE gpa < 2.0;

-- Delete based on subquery
DELETE FROM enrollment
WHERE student_id IN (
    SELECT student_id FROM student WHERE gpa < 2.0
);

-- Delete all rows (but keep table)
DELETE FROM student;  -- Slow (logged, triggers fire)

-- ⚠️ SQLite has no TRUNCATE TABLE command
-- Use DELETE FROM table; instead

-- Drop and recreate (fastest way to empty)
DROP TABLE student;
CREATE TABLE student (...);
```

🧠 Exam Focus: DML Safety

The Golden Rule: Always use WHERE!

```sql
-- ❌ DANGEROUS
UPDATE student SET gpa = 0;          -- Updates ALL students!
DELETE FROM student;                 -- Deletes ALL students!

-- ✅ SAFE
UPDATE student SET gpa = 0 WHERE student_id = 5;
DELETE FROM student WHERE student_id = 5;
```

Safety Tips:

1. Always write the `WHERE` clause FIRST
2. Test with `SELECT` before running `UPDATE` / `DELETE`
3. Use transactions: `BEGIN; ... ROLLBACK;` to undo mistakes
4. Back up your database before bulk operations

Built-in Functions

String Functions in SQLite

```sql
-- Length
SELECT LENGTH('Hello');            -- 5

-- Case conversion
SELECT UPPER('hello');             -- 'HELLO'
SELECT LOWER('HELLO');             -- 'hello'

-- Trimming whitespace
SELECT TRIM('  hello  ');          -- 'hello'
SELECT LTRIM('  hello');           -- 'hello'
SELECT RTRIM('hello  ');           -- 'hello'

-- Substring
SELECT SUBSTR('Hello World', 1, 5);  -- 'Hello'
SELECT SUBSTR('Hello World', 7);     -- 'World'

-- Replace
SELECT REPLACE('Hello World', 'World', 'SQLite');  -- 'Hello SQLite'

-- Concatenation (use || operator)
SELECT 'Hello' || ' ' || 'World';    -- 'Hello World'
```

String Functions: Practical Examples

```sql
-- Full name from first + last
SELECT first_name || ' ' || last_name AS full_name
FROM student;

-- Email domain extraction
SELECT email,
       SUBSTR(email, INSTR(email, '@') + 1) AS domain
FROM student;

-- Search with LIKE (case-insensitive in SQLite)
SELECT * FROM student WHERE first_name LIKE 'a%';   -- Starts with A/a
SELECT * FROM student WHERE last_name LIKE '%maz';   -- Ends with 'maz'
SELECT * FROM student WHERE email LIKE '%@ytu%';     -- Contains '@ytu'

-- INSTR: find position of substring
SELECT INSTR('Hello World', 'World');  -- 7

-- PRINTF: formatted output
SELECT PRINTF('%.2f', 3.14159);  -- '3.14'
SELECT PRINTF('Student: %s, GPA: %.1f', first_name, gpa) FROM student;
```

Numeric Functions in SQLite

```sql
-- Rounding
SELECT ROUND(3.14159, 2);      -- 3.14
SELECT ROUND(3.5);             -- 4.0 (rounds up)
SELECT ROUND(4.5);             -- 4.0 (banker's rounding!)

-- Absolute value
SELECT ABS(-42);               -- 42

-- Min / Max of values
SELECT MIN(10, 20, 5);         -- 5
SELECT MAX(10, 20, 5);         -- 20

-- Random number
SELECT RANDOM();               -- Large random integer
SELECT ABS(RANDOM() % 100);  -- Random 0-99

-- Modulo
SELECT 17 % 5;                 -- 2

-- Type casting
SELECT CAST('42' AS INTEGER);    -- 42
SELECT CAST(3.14 AS INTEGER);   -- 3
SELECT TYPEOF(42);               -- 'integer'
SELECT TYPEOF(3.14);             -- 'real'
SELECT TYPEOF('hello');          -- 'text'
```

Date and Time Functions in SQLite

```sql
-- Current date/time
SELECT date('now');                    -- '2025-03-01'
SELECT time('now');                    -- '14:30:00'
SELECT datetime('now');                -- '2025-03-01 14:30:00'
SELECT julianday('now');               -- Julian day number
SELECT unixepoch('now');               -- Unix timestamp (seconds)

-- Date arithmetic
SELECT date('now', '+7 days');           -- 7 days from now
SELECT date('now', '-1 month');          -- 1 month ago
SELECT date('now', '+1 year');           -- 1 year from now
SELECT date('now', 'start of month');    -- First day of current month
SELECT date('now', 'start of year');     -- Jan 1 of current year

-- Format with strftime
SELECT strftime('%Y', 'now');            -- '2025' (year)
SELECT strftime('%m', 'now');            -- '03' (month)
SELECT strftime('%d', 'now');            -- '01' (day)
SELECT strftime('%H:%M', 'now');         -- '14:30' (time)
SELECT strftime('%w', 'now');            -- '6' (day of week, 0=Sun)
```

Date Functions: Practical Examples

```sql
-- Calculate age from birth_date
SELECT first_name,
       birth_date,
       (strftime('%Y', 'now') - strftime('%Y', birth_date))
       - (strftime('%m-%d', 'now') < strftime('%m-%d', birth_date))
       AS age
FROM student;

-- Students born in a specific year
SELECT * FROM student
WHERE strftime('%Y', birth_date) = '2002';

-- Days until birthday this year
SELECT first_name,
       birth_date,
       julianday(strftime('%Y', 'now') || strftime('-%m-%d', birth_date))
       - julianday('now') AS days_until_birthday
FROM student;

-- Records created in the last 30 days
-- (if you had a created_at column)
SELECT * FROM events
WHERE date(created_at) >= date('now', '-30 days');
```

NULL Handling Functions

```sql
-- COALESCE: return first non-NULL value
SELECT COALESCE(NULL, NULL, 'default');  -- 'default'

-- Practical use: replace NULL with a value
SELECT first_name,
       COALESCE(phone, 'No phone') AS phone
FROM student;

-- IFNULL: SQLite-specific (2 arguments only)
SELECT IFNULL(phone, 'N/A') FROM student;

-- NULLIF: returns NULL if two values are equal
SELECT NULLIF(0, 0);      -- NULL
SELECT NULLIF(42, 0);     -- 42

-- Use NULLIF to avoid division by zero
SELECT total / NULLIF(count, 0) AS average
FROM stats;

-- IIF: inline if (SQLite 3.32+)
SELECT first_name,
       IIF(gpa >= 3.5, 'Honors', 'Regular') AS status
FROM student;
```

CASE Expression

```sql
SELECT first_name, gpa,
    CASE
        WHEN gpa >= 3.7 THEN 'A'
        WHEN gpa >= 3.3 THEN 'A-'
        WHEN gpa >= 3.0 THEN 'B+'
        WHEN gpa >= 2.7 THEN 'B'
        WHEN gpa >= 2.0 THEN 'C'
        ELSE 'F'
    END AS letter_grade
FROM student;

-- Searched CASE
SELECT dept_name,
    CASE dept_name
        WHEN 'Computer Engineering' THEN 'Tech'
        WHEN 'Electrical Engineering' THEN 'Tech'
        WHEN 'Mathematical Engineering' THEN 'Science'
        ELSE 'Other'
    END AS category
FROM department;
```

> 💡 CASE is an **expression** — it returns a value and can be used anywhere a value is expected.

☕ BREAK (15 minutes)

PART 3 — CODING PRACTICE

Building a Complete Database

💻 Practice: E-Commerce Database Schema

Let's build a more realistic database for a semester project example:

```sql
-- Create a fresh database
-- sqlite3 ecommerce.db

PRAGMA foreign_keys = ON;
.headers on
.mode column

-- Customers table
CREATE TABLE customer (
    customer_id  INTEGER PRIMARY KEY,
    first_name   TEXT NOT NULL,
    last_name    TEXT NOT NULL,
    email        TEXT UNIQUE NOT NULL,
    phone        TEXT,
    city         TEXT DEFAULT 'Istanbul',
    created_at   TEXT DEFAULT (datetime('now'))
);
```

💻 Practice: Products & Categories

```sql
    category_id   INTEGER PRIMARY KEY,
    category_name TEXT NOT NULL UNIQUE,
    description   TEXT
);

CREATE TABLE product (
    product_id    INTEGER PRIMARY KEY,
    product_name  TEXT NOT NULL,
    category_id   INTEGER,
    price_cents   INTEGER NOT NULL CHECK (price_cents > 0),
    stock_qty     INTEGER DEFAULT 0 CHECK (stock_qty >= 0),
    is_active     INTEGER DEFAULT 1 CHECK (is_active IN (0, 1)),
    created_at    TEXT DEFAULT (datetime('now')),
    FOREIGN KEY (category_id) REFERENCES category(category_id)
);
```

Design Decisions:

- `price_cents` as INTEGER avoids floating-point errors
- `is_active` as INTEGER simulates BOOLEAN

💻 Practice: Orders & Order Items

```sql
CREATE TABLE orders (  -- "order" is a reserved word, use "orders"
    order_id     INTEGER PRIMARY KEY,
    customer_id  INTEGER NOT NULL,
    order_date   TEXT DEFAULT (datetime('now')),
    status       TEXT DEFAULT 'pending'
                 CHECK (status IN ('pending','processing','shipped','delivered','cancelled')),
    total_cents  INTEGER DEFAULT 0,
    FOREIGN KEY (customer_id) REFERENCES customer(customer_id)
);

CREATE TABLE order_item (
    item_id      INTEGER PRIMARY KEY,
    order_id     INTEGER NOT NULL,
    product_id   INTEGER NOT NULL,
    quantity     INTEGER NOT NULL CHECK (quantity > 0),
    unit_price   INTEGER NOT NULL,  -- price at time of purchase
    FOREIGN KEY (order_id)   REFERENCES orders(order_id),
    FOREIGN KEY (product_id) REFERENCES product(product_id)
);
```

💻 Practice: Insert Sample Data

```sql
-- Categories
INSERT INTO category (category_name, description) VALUES
    ('Electronics', 'Electronic devices and gadgets'),
    ('Books', 'Physical and digital books'),
    ('Clothing', 'Apparel and accessories'),
    ('Sports', 'Sports equipment and gear');

-- Products (prices in cents: $29.99 = 2999)
INSERT INTO product (product_name, category_id, price_cents, stock_qty) VALUES
    ('Wireless Mouse', 1, 2999, 150),
    ('USB-C Hub', 1, 4999, 75),
    ('Python Crash Course', 2, 3499, 200),
    ('SQL Pocket Guide', 2, 1999, 120),
    ('Running Shoes', 3, 8999, 50),
    ('Cotton T-Shirt', 3, 1499, 300),
    ('Yoga Mat', 4, 2499, 80),
    ('Dumbbell Set', 4, 5999, 40);
```

💻 Practice: Insert Customers & Orders

```sql
-- Customers
INSERT INTO customer (first_name, last_name, email, city) VALUES
    ('Ahmet', 'Yılmaz', 'ahmet@email.com', 'Istanbul'),
    ('Ayşe',  'Kara',   'ayse@email.com',  'Ankara'),
    ('Mehmet','Demir',  'mehmet@email.com', 'Izmir'),
    ('Zeynep','Çelik',  'zeynep@email.com', 'Istanbul'),
    ('Can',   'Öz',     'can@email.com',    'Bursa');

-- Orders
INSERT INTO orders (customer_id, order_date, status, total_cents) VALUES
    (1, '2025-01-15 10:30:00', 'delivered', 7998),
    (1, '2025-02-20 14:15:00', 'shipped', 3499),
    (2, '2025-02-25 09:00:00', 'processing', 10998),
    (3, '2025-03-01 11:45:00', 'pending', 2999),
    (4, '2025-01-10 16:20:00', 'delivered', 14998);

-- Order items
INSERT INTO order_item (order_id, product_id, quantity, unit_price) VALUES
    (1, 1, 2, 2999),   -- 2x Wireless Mouse
    (2, 3, 1, 3499),   -- 1x Python Crash Course
    (3, 5, 1, 8999),   -- 1x Running Shoes
    (3, 4, 1, 1999),   -- 1x SQL Pocket Guide
    (4, 1, 1, 2999),   -- 1x Wireless Mouse
    (5, 5, 1, 8999),   -- 1x Running Shoes
    (5, 7, 1, 2499),   -- 1x Yoga Mat
    (5, 6, 2, 1499);   -- 2x Cotton T-Shirt
```

💻 Practice: Query Exercises

```sql
-- 1. All products with price displayed in dollars
SELECT product_name,
       PRINTF('$%.2f', price_cents / 100.0) AS price,
       stock_qty
FROM product
ORDER BY price_cents DESC;

-- 2. Products that are low in stock (< 60)
SELECT product_name, stock_qty
FROM product
WHERE stock_qty < 60
ORDER BY stock_qty;

-- 3. Active products in Electronics category
SELECT p.product_name, p.stock_qty
FROM product p
WHERE p.category_id = 1 AND p.is_active = 1;

-- 4. Customer list sorted by city
SELECT first_name || ' ' || last_name AS full_name, city
FROM customer
ORDER BY city, last_name;
```

Practice: Aggregate Queries

```sql
-- 5. Total number of products
SELECT COUNT(*) AS total_products FROM product;

-- 6. Average price per category
SELECT c.category_name,
       COUNT(*) AS num_products,
       PRINTF('$%.2f', AVG(p.price_cents) / 100.0) AS avg_price
FROM product p, category c
WHERE p.category_id = c.category_id
GROUP BY c.category_name;

-- 7. Total stock value
SELECT PRINTF('$%.2f', SUM(price_cents * stock_qty) / 100.0) AS total_stock_value
FROM product;

-- 8. Most expensive product
SELECT product_name,
       PRINTF('$%.2f', price_cents / 100.0) AS price
FROM product
ORDER BY price_cents DESC
LIMIT 1;
```

Practice: String Function Exercises

```sql
-- 9. Uppercase product names
SELECT UPPER(product_name) AS product_upper
FROM product;

-- 10. Customer email domains
SELECT first_name,
       email,
       SUBSTR(email, INSTR(email, '@') + 1) AS email_domain
FROM customer;

-- 11. Product name lengths
SELECT product_name,
       LENGTH(product_name) AS name_length
FROM product
ORDER BY name_length DESC;

-- 12. Search products containing 'S'
SELECT product_name FROM product
WHERE product_name LIKE '%s%';

-- 13. Format customer display
SELECT PRINTF('%-15s %-15s %s', first_name, last_name, city) AS formatted
FROM customer;
```

Practice: Date Function Exercises

```sql
-- 14. Orders placed in February 2025
SELECT * FROM orders
WHERE order_date >= '2025-02-01' AND order_date < '2025-03-01';

-- 15. Days since each order was placed
SELECT order_id,
       order_date,
       CAST(julianday('now') - julianday(order_date) AS INTEGER) AS days_ago
FROM orders;

-- 16. Orders by month
SELECT strftime('%Y-%m', order_date) AS month,
       COUNT(*) AS order_count
FROM orders
GROUP BY month
ORDER BY month;

-- 17. Customer account age (days since creation)
SELECT first_name,
       created_at,
       CAST(julianday('now') - julianday(created_at) AS INTEGER) AS account_age_days
FROM customer;
```

💻 Practice: CASE and Conditional Logic

```sql
-- 18. Categorize products by price range
SELECT product_name,
       PRINTF('$%.2f', price_cents / 100.0) AS price,
       CASE
           WHEN price_cents >= 5000 THEN 'Premium'
           WHEN price_cents >= 2500 THEN 'Standard'
           ELSE 'Budget'
       END AS price_tier
FROM product
ORDER BY price_cents DESC;

-- 19. Order status with emoji
SELECT order_id,
       CASE status
           WHEN 'delivered'  THEN '✅ Delivered'
           WHEN 'shipped'    THEN '📦 Shipped'
           WHEN 'processing' THEN '⚙️ Processing'
           WHEN 'pending'    THEN '⏳ Pending'
           WHEN 'cancelled'  THEN '❌ Cancelled'
       END AS display_status
FROM orders;
```

💻 Practice: UPDATE Exercises

```sql
-- 20. Increase price of Electronics by 10%
UPDATE product
SET price_cents = CAST(price_cents * 1.10 AS INTEGER)
WHERE category_id = 1;

-- Verify
SELECT product_name, PRINTF('$%.2f', price_cents / 100.0) AS new_price
FROM product WHERE category_id = 1;

-- 21. Mark delivered orders as 'completed'
-- First, add the status to CHECK constraint
-- (In SQLite, we'd need to recreate the table or just use the value)

-- 22. Update stock after an order
UPDATE product
SET stock_qty = stock_qty - 1
WHERE product_id = 1;

-- 23. Deactivate out-of-stock products
UPDATE product
SET is_active = 0
WHERE stock_qty = 0;
```

💻 Practice: ALTER TABLE in SQLite

```sql
-- Add a new column
ALTER TABLE customer ADD COLUMN loyalty_points INTEGER DEFAULT 0;

-- Rename a table
ALTER TABLE customer RENAME TO customers;
ALTER TABLE customers RENAME TO customer;

-- Rename a column (SQLite 3.25+)
ALTER TABLE customer RENAME COLUMN phone TO phone_number;

-- ⚠️ SQLite limitations:
-- Cannot DROP a column (before SQLite 3.35)
-- Cannot change a column's type
-- Cannot add constraints to existing columns

-- Workaround: Recreate the table
-- 1. Create new table with desired schema
-- 2. Copy data: INSERT INTO new_table SELECT ... FROM old_table
-- 3. Drop old table
-- 4. Rename new table
```

Constraints Summary for Exam

`PRIMARY KEY`	Unique row ID, NOT NULL	`id INTEGER PRIMARY KEY`
`NOT NULL`	No NULL values	`name TEXT NOT NULL`
`UNIQUE`	No duplicate values	`email TEXT UNIQUE`
`CHECK`	Validate condition	`CHECK (age >= 18)`
`DEFAULT`	Fallback value	`DEFAULT 'active'`
`FOREIGN KEY`	Referential integrity	`REFERENCES dept(id)`

Inline vs Table-Level:

```sql
-- Inline (column-level)
CREATE TABLE t (id INTEGER PRIMARY KEY, val INTEGER CHECK (val > 0));

-- Table-level (required for composite constraints)
CREATE TABLE t (a INT, b INT, PRIMARY KEY (a, b), CHECK (a != b));
```

Practice Exercises (Homework)

1. **Extend the e-commerce database:**
   - Add a `review` table (review_id, product_id FK, customer_id FK, rating CHECK 1-5, comment, created_at)
   - Add a `shipping_address` table linked to customers
   - Insert sample data
2. **Write these queries:**
   - Find all products cheaper than the average price
   - Count orders per customer
   - Find customers who have never placed an order
   - Calculate the total revenue per product category
   - Find the most popular product (most ordered)
3. **Use string/date functions to:**

🔮 Next Week Preview

Week 3: Advanced Queries

- **JOINs:** INNER, LEFT, RIGHT, CROSS, Self
- **GROUP BY** with aggregate functions
- **HAVING** vs WHERE
- **UNION** and set operations
- **Subqueries** and derived tables
- **SQL Logical Execution Order**

Preparation:

- 📖 Read: *Learning SQL* Ch. 3–5
- 💻 Complete this week's homework exercises
- 🎓 Watch: CS50 SQL Week 1 — Relating

Thank You!

Questions?

Next class: Advanced Queries — JOIN, UNION, GROUP BY & HAVING

📧 Contact: Course LMS / Email

📚 *Learning SQL* (Beaulieu, O'Reilly)

🎓 CS50 SQL — cs50.harvard.edu/sql